Data Vault 2.0 Modeling Specification

With Real-World Examples and Diagrams

Covers all entity types, table classifications, field elements,
Hub, Link and Satellite rules, naming conventions, and deprecated patterns.

Includes supplements from Hans Hultgren's Data Vault Modeling Guide (2012)

Based on Data Vault Data Modeling Specification v2.0.2
Dan Linstedt, 2018

Ch 1

What is Data Vault? — The Big Picture

Data Vault is a data warehouse modeling approach designed for **agility, auditability, and scalability**. It separates three concerns: **what things exist** (Hubs), **how they relate** (Links), and **what they look like over time** (Satellites).

Component	Analogy	Real-World Example
Hub	A register of unique entities — like a phone book listing names only	HUB_CUSTOMER stores every unique Customer Number ever seen in any source system
Link	A record of relationships — like a marriage certificate connecting two people	LINK_ORDER records each unique Customer–Product–Order combination
Satellite	A diary of changes — like a medical record noting every visit over time	SAT_CUSTOMER_DETAILS stores name, address, email — new row every time they change

■ Supermarket Analogy

Think of a supermarket: HUB_PRODUCT lists every product ever stocked. LINK_SALE records every unique sale connecting a customer, product, and store. SAT_PRODUCT_DETAILS records name, price, and category — and every time they change a new row is added rather than the old one being overwritten.

Approach	Description
Data Vault (Raw Vault layer)	Stores RAW data exactly as it came from source — no business rules applied. Designed for fast, parallel loading. Fully auditable. Flexible to add new sources.
Kimball Star Schema (Information Mart layer)	Stores business-ready data in fact and dimension tables — business rules applied during ETL. Designed for fast queries and ease of use by analysts.
In practice	Many organisations use Data Vault as the integration and storage layer, then build Kimball-style star schemas on top of it for reporting and BI dashboards.

Ch 1 ■ Supplement

The EDW Program & the Backbone (Hultgren)

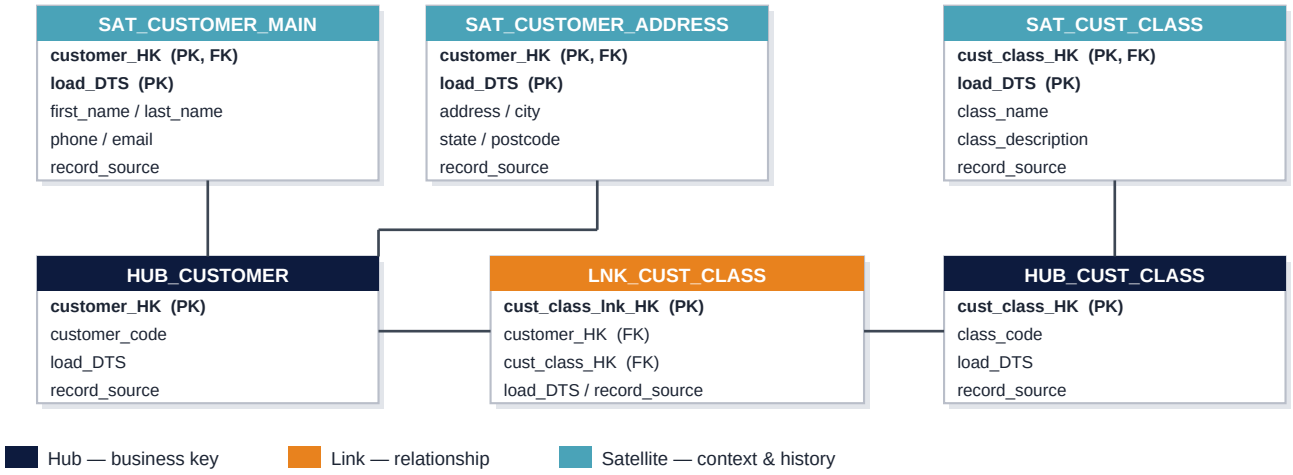
Hultgren frames Data Vault first and foremost as the modeling approach for an Enterprise Data Warehouse PROGRAM — not a project. A project has a beginning, an end and fixed goals; the EDW has none of these. It is an ongoing capability that continuously absorbs change, which is exactly the environment the Hub / Link / Satellite separation is designed for.

The EDW is a program, not a project — it continuously absorbs four kinds of change

Incremental activity	Examples
a) New data sources	Internal new systems, external integrations, acquisitions
b) Changes to existing sources	New tables, new attributes, new domain values, new formats, new rules
c) New business rules on keys	Changes to alignment, grain, cardinality and domain values of business keys — and to the relationships between them
d) New downstream requirements	New subject areas, new business rules, regulatory / compliance reporting, changes to operational latency

Applied consistently, the reward is auditability, agility, adaptability, alignment with the business, and support for operational data warehousing. The cost is commitment: to the EDW program itself, and to the consistency and integrity of the three core constructs above all else.

The backbone (skeletal structure) — Hubs + Links carry keys; Satellites carry everything else



■ All context, all history — in Satellites only

The Hub-and-Link backbone holds a strict 1:1 picture of core business concepts and their natural relationships, with no descriptive data and no dates of business meaning. Hultgren's 2012 guide keys these tables with sequence IDs (H_Customer_SID); shown here with hash keys (_HK) per the DV 2.0 specification in this guide — see Ch 9, where sequence IDs are deprecated.

Ch 2

Entity Type Definitions — Hubs, Links, Satellites

1.1 Hub Entity

A Hub is a **unique list of business keys**. It does NOT store descriptions, history, or relationships — just the keys. Think of it as a telephone directory that lists every unique customer number ever seen.

Column	Example Value	Description
hub_customer_hash_key	HK-a3f9...	Hash of the business key (optional surrogate)
customer_number	CUST-10045	The actual business key — the customer ID from the source
load_date_timestamp	2024-06-01 08:00:00	When this key was first seen in the warehouse
record_source	CRM_SYSTEM	Which source system provided this business key

■ Analogy

A Hub is like the index at the back of a book — it just lists every unique entity that exists (every customer number, every product code) with no extra detail. All descriptive detail lives in the Satellites attached to it.

1.1.1 Stub Hub

A Stub Hub is a placeholder Hub — the business key is known but the source system or descriptive data is out of scope for the current sprint. It reserves the slot in the model so Links can connect to it now, and details can be filled in later.

Scenario	What Exists Now	What Changes Later
Sprint 1: build sales	HUB_SUPPLIER is created with supplier codes only	SAT_SUPPLIER_DETAILS not built yet — out of scope for this sprint
Sprint 3: add supplier detail	SAT_SUPPLIER_DETAILS is now built and attached to the existing Hub	Nothing missing — the stub becomes a full Hub with descriptive context

1.2 Link Entity

A Link records a **unique list of relationships between two or more business keys**. Links are **never temporal** — no begin or end dates (one exception: non-historized links). Temporality always lives in Effectivity Satellites attached to the Link.

Column	Example Value	Description
link_order_item_hash_key	HK-b7c2...	Composite hash of all imported Hub keys
hub_customer_hash_key	HK-a3f9...	Foreign key to HUB_CUSTOMER
hub_product_hash_key	HK-d1e5...	Foreign key to HUB_PRODUCT
hub_order_hash_key	HK-f8g3...	Foreign key to HUB_ORDER
load_date_timestamp	2024-06-01 09:15:00	When this relationship was first seen
record_source	ORDER_SYSTEM	Source system that produced this relationship

1.2.1 Hierarchical Link

Connects the SAME Hub to itself — representing parent-child relationships within one entity type. For example a product belonging to a product bundle, or an employee reporting to a manager.

Example	Parent Key	Child Key	Meaning
Product bundle	HK — Product Bundle A	HK — Individual Item 1	Item 1 belongs to Bundle A — same HUB_PRODUCT referenced twice
Manager-report	HK — Manager Employee 7	HK — Direct Report Employee 12	Employee 12 reports to Employee 7 — same HUB_EMPLOYEE referenced twice

1.2.2 Same-As Link

Maps synonyms — different business keys in different source systems that refer to the same real-world entity. Maps aliases to a single master key.

Master Key	Alias Key	Alias Source	What It Means
CUST-001 (master)	00045	ERP System	Both refer to the same customer Alice Brown — mapped to master key
PROD-ABC (master)	SKU-9901	Warehouse System	Both refer to the same product Organic Banana — different system codes

1.2.3 Non-Historized Link

Stores immutable (never-changing) facts directly on the link alongside the keys. Used for completed transactions that will never be updated — the one exception to the no-dates-on-links rule.

Column	Example Value	Notes
link_transaction_hash_key	HK-t901...	Primary key — hash of all relationship keys
hub_account_hash_key	HK-acc01	FK to HUB_ACCOUNT
hub_merchant_hash_key	HK-mer05	FK to HUB_MERCHANT
transaction_amount	$54.90	Immutable fact — will NEVER change after this row is inserted
transaction_date	2024-06-01	Date is allowed here because this is a non-historized (immutable) link

1.2.4 Exploration Link (Deep Learning Link)

Built for a specific data science or machine-learning purpose. A throw-away link — can be built and discarded without affecting auditability. Typically holds strength and confidence scores from neural network analyses.

Column	Example Value	Meaning
hub_customer_hash_key	HK-a3f9	Customer being analysed
hub_product_hash_key	HK-d1e5	Product being analysed
strength_score	0.87	87% — how strong the correlation is between this customer and product

Column	Example Value	Meaning
confidence_score	0.91	91% — how confident the model is in that strength prediction

1.3 Satellites

Satellites store **delta-driven descriptive information** — data that changes over time. Every time an attribute changes, a new row is inserted (never updated). Satellites are split by rate of change or type of data so fast-changing and slow-changing attributes live in separate satellite tables.

SAT_CUSTOMER_CONTACT — fast-changing attributes: email, phone

Column	Example Value	Description
hub_customer_hash_key	HK-a3f9...	FK to parent HUB_CUSTOMER
load_date_timestamp	2024-06-01 08:00:00	Part of PK — identifies this version
record_source	CRM_SYSTEM	Where this version came from
email_address	alice@example.com	Descriptive attribute — changes occasionally
phone_number	+61 400 111 222	Descriptive attribute — changes occasionally

SAT_CUSTOMER_DEMOGRAPHICS — slow-changing attributes: date of birth, gender

Column	Example Value	Description
hub_customer_hash_key	HK-a3f9...	Same parent customer — same Hub FK
load_date_timestamp	2020-01-15 00:00:00	Loaded once at onboarding — rarely needs updating
date_of_birth	1985-03-22	Changes almost never — kept separate from fast attributes
gender	Female	Changes almost never — no need to duplicate in contact satellite

■ Why Split Satellites?

Splitting satellites by rate of change is critical for performance. If email changes weekly but date of birth never changes, putting them together means every email update creates a new row that duplicates the unchanged date of birth — wasting storage and slowing delta detection. Keep them separate.

1.3.1 Multi-Active Satellite

Built for entities with multiple simultaneously-active child records that have no standalone business key of their own. Example: an employee with three active addresses at the same time — home, work, and postal.

hub_employee_hk	load_date_ts	address_seq	address_type	address_line_1
HK-emp01	2024-06-01	1	Home	12 Smith Street, Sydney
HK-emp01	2024-06-01	2	Work	100 Market Street, Sydney CBD
HK-emp01	2024-06-01	3	Postal	PO Box 445, Sydney

1.3.2 Effectivity Satellite

Stores begin and end dates for a Hub or Link record — recording WHEN a relationship or business key was active. This is the ONLY place in Data Vault where temporal start and end dates are stored. Physical updates are never allowed.

parent_hash_key	load_date_ts	effective_from	effective_to	is_deleted
HK-order01 (link)	2024-01-01	2024-01-01	2024-03-31	N — relationship was active Jan to Mar
HK-order01 (link)	2024-04-01	2024-04-01	9999-12-31	N — reactivated April, currently active
HK-cust99 (hub)	2024-05-15	2024-05-15	9999-12-31	Y — customer was deleted in source system

1.3.3 System Driven Satellite

Populated by automated business rules rather than source data feeds. Point-in-Time and Bridge tables are examples. Can be dropped and rebuilt at any time without affecting auditability of the underlying raw data.

Type	What It Contains	Purpose
Point-in-Time (PIT)	Snapshot of all Satellite keys for one Hub at a given moment in time	Enables fast equal-join queries without scanning full Satellite history
Bridge Table	Keys spread across multiple Hubs and Links in one pre-joined snapshot structure	Acts as a base-level fact table — pre-joins common key structures for query performance
Record Source Tracking	Counts and metadata about record sources observed over time	Monitoring and data quality tracking — can be rebuilt if dropped

Ch 2 ■ Supplement

Modeling Process & Thinking Differently (Hultgren)

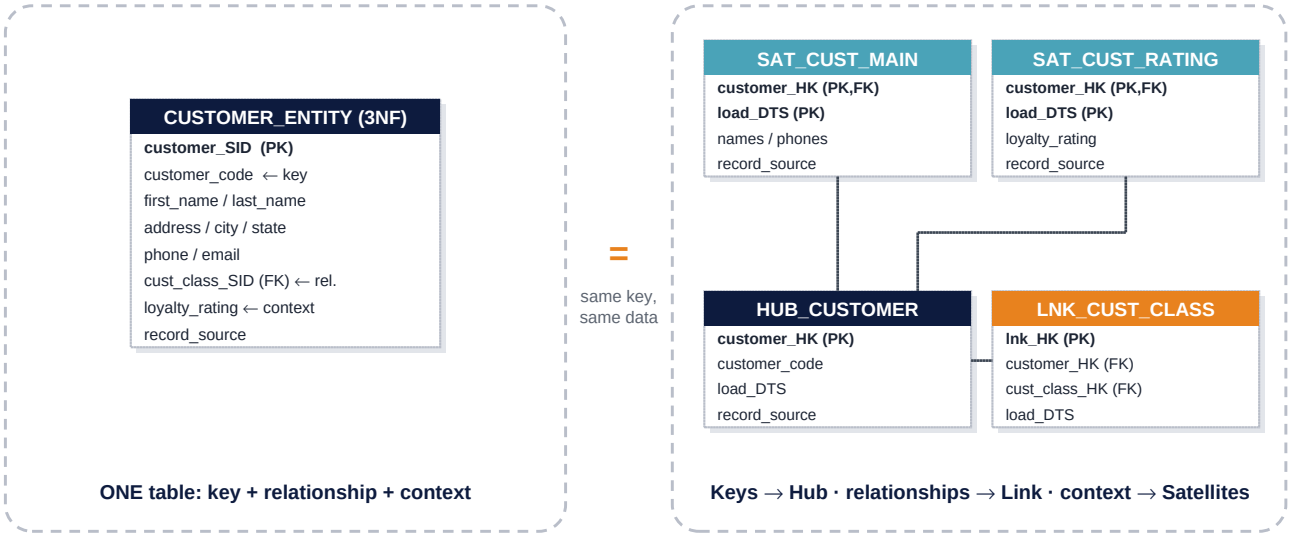
Modeling with the Data Vault — a business-analysis-driven, 3-phase process

Step	Task	Phase
1.1	Identify Business Concepts	Hubs
1.2	Establish Enterprise-Wide Business Keys (EWBK) for Hubs	Hubs
1.3	Model Hubs	Hubs
2.1	Identify Natural Business Relationships	Links
2.2	Analyze the Relationship's Unit of Work	Links
2.3	Model Links	Links
3.1	Gather Context Attributes that Describe the Keys	Satellites
3.2	Establish Criteria & Design the Satellites	Satellites
3.3	Model Satellites	Satellites

The process deliberately ignores the fact-vs-dimension and master-vs-transaction distinctions: events and transactions are Hub candidates exactly like Customer or Product. The only question is “what are the core business concepts, and what are their unique business keys?”

Think differently — the same information, separated instead of combined

In 3NF (and in a conformed dimension) the business key, the relationships (FKs) and all context attributes live in ONE table. Data Vault represents exactly the same information — the same single business key, the same attributes, the same relationship — but separates the three concerns:



■ HINT — when you stop vaulting

We tend to see tables the way we always have, and re-combine keys with relationships with context. The moment you change a Satellite's grain, or put a relationship FK into a Satellite or Hub, you have stopped vaulting and returned to other forms of modeling. Before doing so, redraw the two circles above and reconsider: the only grain shift inside the circle should be the load date that tracks history.

Ch 3

Other Table Classifications

2.1 Stand-Alone Tables

Immutable reference data that never changes and has no history. Calendar and time tables are the classic examples — everyone agrees what Monday means and June has 30 days.

Table	Contents	Does It Ever Change?
CALENDAR_DATE	date, day_name, month_name, fiscal_period, is_holiday	Never — what happened on 1 June 2024 is forever fixed
TIME_OF_DAY	hour, minute, am_pm, day_part (morning, afternoon, evening)	Never — 14:30 always means 2:30pm
COUNTRY_CODE	ISO country codes and country names	Extremely rarely — new countries are almost never created

2.2 Reporting Tables

Flat, wide, denormalized structures in the Information Mart layer — the equivalent of Kimball star schema tables. Business rules have been applied before loading.

Feature	Reporting Table	Raw Data Vault Table
Business rules	Yes — soft rules transform raw data for business use	No — raw data preserved exactly as received from source
Layer	Information Mart — the presentation layer	Raw Vault — the integration and storage layer
Structure	Flat wide denormalized — like a large Excel spreadsheet	Normalized into Hub, Link, Satellite components
Queried by	Business users, Power BI, Tableau	ETL processes and data engineers

2.3 Point-in-Time and Bridge Tables

Performance structures that pre-join keys across Satellites and Hubs or Links. They are System Driven Satellites — derived structures that can be rebuilt without affecting the underlying raw data.

Table Type	What It Does	Real-World Benefit
Point-in-Time (PIT)	Takes a snapshot of all Satellite effective keys for one Hub or Link at specific moments in time	Instead of joining 5 Satellites with complex date logic, join once to the PIT table and get the right keys instantly
Bridge Table	Combines primary keys across multiple Hubs and Links into a single snapshot	Acts like a pre-built fact table skeleton — analysts join to it instead of navigating Hub, Link, and Satellite structures manually

2.4 Reference Tables

Code and description lookup structures — like a postcode list or category codes. Constructed using standard Hub, Link, and Satellite entities. No physical foreign keys — references are implied and resolved at query time.

Example Reference Table	Codes Stored In	Descriptions Stored In
Product Category Codes	HUB_CATEGORY (category_code as the business key)	SAT_CATEGORY_DESC (category_name and description)
State and Territory Codes	HUB_STATE (state_code as the business key)	SAT_STATE_DESC (state_name and country)
Payment Method Codes	HUB_PAYMENT_METHOD (method_code as the business key)	SAT_PAYMENT_METHOD_DESC (method_name and description)

2.5 Staging Tables

Temporary holding structures between the source system and the Raw Data Vault. Two purposes: bulk load performance, and parallel downstream loading. A second-level staging table may pre-compute hash keys and load dates.

Stage Level	Contents	Purpose
Level 1 Staging	Raw data exactly as extracted from source — no transformations applied	Bulk load performance — fast inserts from source files or streams
Level 2 Staging (optional)	Primary key of stage record plus computed system fields: hash keys, hash differences, load dates, and record sources	Parallel loading — ETL computes heavy hash operations once then fans out to multiple vault tables simultaneously

2.6 Landing Zone Files — NoSQL Environments

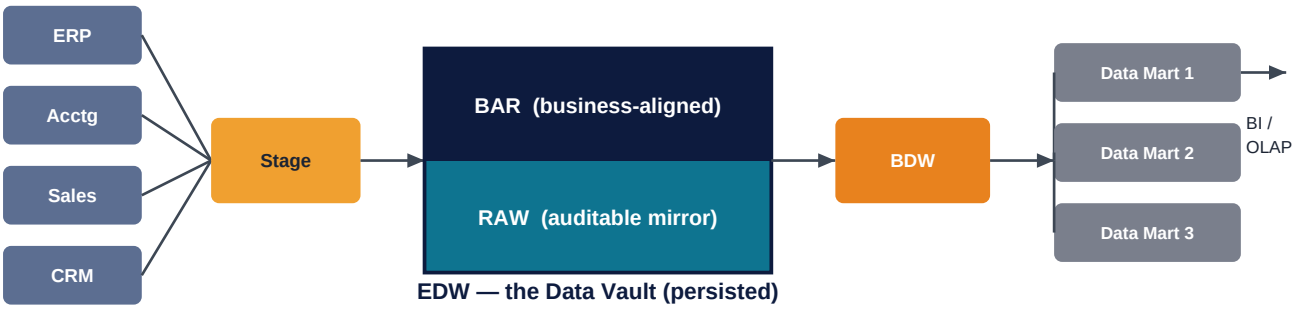
In a Hadoop or cloud object storage environment, raw files dropped into a managed storage area such as HDFS, S3, or Azure Data Lake. The metadata engine registers files and enables parallel partitioned access.

Characteristic	What It Means in Practice
Parallel access	Multiple ETL processes can read different partitions of the same file simultaneously — much faster than sequential processing
Partitioning and sharding	Files are split by date, source, or key range for efficient parallel processing
Metadata registration	File metadata registered with Hadoop HCatalog or equivalent. Load date is stamped at the file level rather than at the individual row level

Ch 3 ■ Supplement

DV EDW Architecture, Persistence & Sysgen (Hultgren)

Hultgren's high-level architecture places one persisted EDW at the centre, holding both the RAW (auditable, source-mirroring) and the business-aligned components, with disposable layers either side:



RAW and BAR are a logical designation — they may be physically separate or live in one physical layer with Links managing the alignment between raw keys and business keys (see Page 13A).

What is persisted — and what is deliberately disposable

Layer	Persisted?	Notes
Stage	No	Overwritten each load — a transient landing area only (Ch 3, §2.5)
EDW — Raw Vault	Yes	The system of record: all data, atomic, fully traceable to source
BDW / Business Vault	Commonly	Persisting it is common but not required — it can be re-derived
Data Marts	No	Rebuilt from the vault on demand — a key difference from dimensional (federated) warehouses, which persist their facts and dimensions

■ "Sysgen" — tables we generate, clearly labelled

Any table loaded by business-rule processing rather than straight from a source — PIT and Bridge tables, business-aligned Hubs, derived Satellites — must carry a record_source of SYSGEN (system generated). They exist for performance and presentation only; the raw, auditable data used to build them remains the sole long-term source of truth. This is the same rule this guide applies to System-Driven Satellites in Ch 2 §1.3.3 and PIT / Bridge tables in Ch 3 §2.3.

Ch 4 Common Field Elements — System Metadata Fields

These fields are system-driven and system-managed — generated automatically during ETL loading. They are NOT business data and do NOT fall under business audit. They exist to enable traceability, parallel loading, and delta detection.

3.1 Hash Key (Optional)

A deterministic value computed from the business key using a hashing function like MD5 or SHA-256. Given the same business key input you always get the same hash output. This enables parallel loading without sequential foreign key lookups.

Entity	Business Key Input	Hash Key Output	Why Hash Instead of Sequence?
Customer	CUST-10045	0x3A7F...B2C1 (MD5)	Any ETL process can compute this independently — no central lookup table needed
Product	PROD-ABC-500g	0xF1D2...9E33	Enables parallel inserts into Hub, Link, and Satellite simultaneously
Order	ORD-20240601-99	0x8B4A...C7E0	Consistent across geographic splits and hybrid cloud or on-premise environments

3.2 Business Key (Required)

The natural identifier from the source system that the business uses to find records. Must have the same meaning across all lines of business. If two systems call different things by the same key name they need separate Hubs.

Key Type	Example	Valid Business Key?
Natural key	Customer Number CUST-10045	Yes — used by the business to look up customers in the source system
Sequence shown to users	Invoice Number INV-0099 printed on all invoices	Yes — once shown to business users it becomes their business key
Internal DB sequence	Auto-increment ID 99 never shown to users	No — this is a database artefact, not a business identifier

3.3 Hash Difference (Optional)

A hash computed over ALL descriptive columns in a Satellite. Instead of comparing 20 columns one by one to detect a change, compare just the single hash difference column.

Approach	What the ETL Does	Performance Impact
Without Hash Difference	Compare: email AND phone AND address AND loyalty_tier AND last_purchase_date AND ... (20 separate column comparisons per row)	Slower — more CPU and I/O per row as data volumes grow
With Hash Difference	Compare: hash_diff column only. If old hash_diff differs from new hash_diff then something changed — load the new Satellite row	Faster — one comparison per row regardless of how many columns exist

3.4 Load Date Time Stamp (Required in RDBMS, Optional in NoSQL)

The exact date and time a record was inserted into the Data Vault. Part of the primary key in Satellites. Never part of the PK in Hubs or Links.

Table Type	Role of Load Date Time Stamp	Part of Primary Key?
Hub	Records the FIRST time this business key was ever seen — never changes after initial load	No — attribute only. The Hub PK is the hash key of the business key only
Link	Records the FIRST time this specific relationship was ever observed	No — attribute only. Link PK is the composite of all imported Hub keys
Satellite	Identifies WHICH VERSION of the data this row represents — the row effective date. Without it you cannot distinguish versions	Yes — Hub or Link FK plus Load Date together form the Satellite PK

3.5 Record Source (Required)

The name of the source system that generated the data. Stamped automatically on every row at load time. Provides full row-level traceability back to source.

Record Source Value	Meaning
CRM_SYSTEM	This customer record came from Salesforce CRM
ORDER_SYSTEM	This order relationship came from the order management system
ERP_SAP	This financial record came from SAP ERP
MANUAL_LOAD_20240601	This record was manually loaded on 1 June 2024 — useful for tracking corrections

3.6 Update User and Update Time Stamp (Optional — Metrics Vault Only)

DBA-level tracking fields recording who modified a record and when. These must NEVER appear in the Raw Data Vault because they require physical UPDATE statements which do not scale to large data volumes.

Field	Where It Lives	Why NOT in Raw Vault?
Update User	Metrics Vault component only	Requires a physical UPDATE statement on existing rows — not scalable at large data volumes and not supported by all platforms
Update Time Stamp	Metrics Vault component only	Same reason — physical updates on hundreds of terabytes are impractical and break the parallelism that Data Vault relies on

3.7 Extract Date (Optional)

Records when the data was extracted from the source system. Different from load date: extract date is when the source produced the data, load date is when the vault received and inserted it.

Date Field	What It Records	Example Value
Extract Date	When the source system produced or exported the data	Source ran its nightly extract at 23:00 on 31 May 2024
Load Date Time Stamp	When the Data Vault pipeline actually inserted the row	ETL loaded this record at 02:15 on 1 June 2024
Gap between them	Processing and transit time through the ETL pipeline	35 minutes elapsed between extract and load

3.8 Applied Date Time Stamp (Optional)

A business-meaningful timestamp indicating WHEN in the business timeline the data applies. Used for real-time feeds (applied = transaction creation time) or historical backdated corrections (applied = the past date when data should be effective).

Use Case	Load Date (when vault received it)	Applied Date (when business event occurred)
Real-time transaction	2024-06-01 14:32:05 — now, when vault received it	2024-06-01 14:31:58 — 7 seconds earlier when transaction was actually created
Late arriving historical data	2024-06-10 02:00:00 — today, when data finally arrived	2024-01-15 09:00:00 — the actual business date the data should apply to

Ch 5

Hub Rules — 4.0

These are the governing rules for designing and loading Hubs. Every Hub must comply with all applicable rules.

4.1 — Hub Must Have at Least One Business Key

Every Hub must have at least one business key field. Without a business key there is nothing to identify the entity. The business key IS the Hub.

Valid Hub Design	Invalid Hub Design
HUB_CUSTOMER with customer_number as the business key column	A table with only a hash key and load date — no business key means no entity identity

4.2 — Business Key to Surrogate is One-to-One

If a hash key or sequence surrogate is used it must be one-to-one with the business key. One business key value maps to exactly one and only one surrogate.

Business Key	Hash Key	Valid?
CUST-10045	HK-a3f9...	Yes — one business key maps to exactly one hash every time
CUST-10045 entered twice	Two different hashes produced	No — the same business key cannot produce two different surrogates

4.3 — Hub Cannot Contain Multiple Stand-Alone Business Keys

If two business keys are independently meaningful they CANNOT live in the same Hub. Each gets its own Hub, connected by a Link entity.

Wrong Design	Correct Design
One Hub with BOTH customer_number AND invoice_number as separate columns	HUB_CUSTOMER (customer_number only) plus HUB_INVOICE (invoice_number only) plus LINK_CUSTOMER_INVOICE connecting the two Hubs

4.4 — Hubs Should Have at Least One Satellite

A Hub without any Satellite has no context — just a list of keys with no description. A Stub Hub is the correct pattern when detail is temporarily out of scope.

Hub State	What It Means
Hub with Satellites attached	Complete — business key plus descriptive context that changes over time
Hub without Satellites — Stub Hub	Placeholder — source system or detail is out of scope for this sprint
Hub without Satellites permanently	Problem — likely indicates bad source data or a missing source system in scope

4.5 — Composite Key When Source Systems Collide

A single Hub can have a composite business key ONLY when two source systems use identical key values to mean completely different things. The source system name becomes part of the composite to guarantee uniqueness.

System A Value	System B Value	Collision Problem	Composite Key Solution
PROD-001 = Banana in System A	PROD-001 = Laptop in System B	Integrating both creates a false match — same key but different entities	PROD-001+SYSTEM_A and PROD-001+SYSTEM_B become two distinct composite keys

4.6 — Business Key Stands Alone

The business key must be independently meaningful. A sequence ID shown to business users qualifies because the business uses it to look up records.

Key Type	Standalone?	Qualifies as Business Key?
Customer Number CUST-10045	Yes — used independently by the business	Yes — business references it directly
Invoice Number INV-0099 printed on invoices	Yes — business refers to it daily	Yes — shown to and used by business users
Auto-increment DB row ID never shown to users	No — internal database artefact only	No — this is a database implementation detail not a business identifier

4.7 — Load Date is an Attribute, Never Part of Hub Primary Key

The Hub load date records the FIRST time the business key arrived in the warehouse. It is an attribute only — if it were in the PK the same business key arriving on two different days would create two Hub rows for the same entity.

Hub Primary Key Design	Correct?
hub_customer_hash_key — only the hash of the business key	Yes — unique per business key. Load date stored as an attribute alongside it
hub_customer_hash_key PLUS load_date_timestamp combined as PK	No — same customer seen on two different dates would create two rows for the same customer. One business entity = one Hub row always

4.8 — Record Source is NOT Part of Hub Primary Key

If two source systems report the same customer it is still ONE customer — the Hub gets one row. Record source records which system was FIRST to report it.

Scenario	Rows in Hub	Record Source Field
CRM reports CUST-10045 on 1 January	1 row inserted	record_source = CRM_SYSTEM — the first system to report this key
ERP also reports CUST-10045 on 5 January	Still 1 row — no duplicate inserted	record_source unchanged — CRM was first. The ERP observation is tracked elsewhere in audit mechanisms

Ch 5 ■ Supplement

The Business Key — EWBK & Key Alignment (Hultgren)

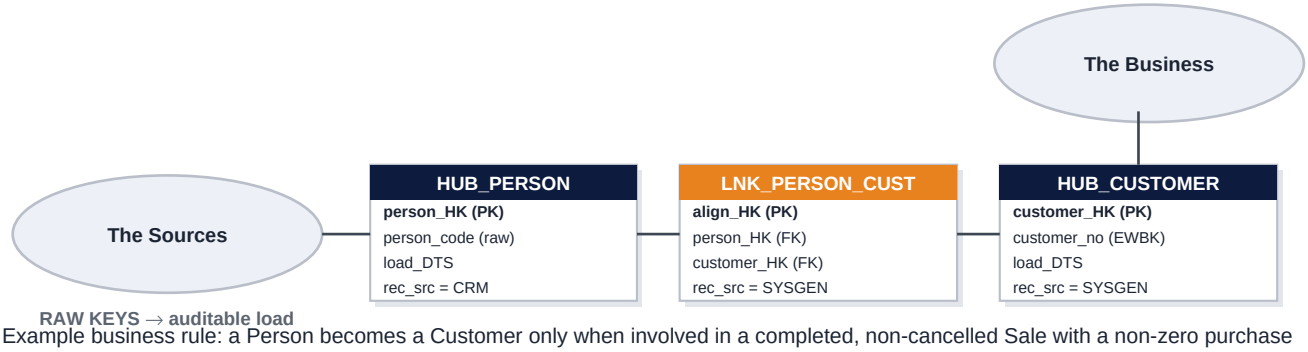
Rule 4.1 says the business key IS the Hub. Hultgren adds the enterprise dimension: because the DV program is organisational in scope, Hub keys should be Enterprise-Wide Business Keys (EWBKs) — keys that are meaningful across the whole organisation, transcend time, and survive the replacement of any individual source system. Designing them is closer to business requirements gathering than to source-system analysis: with hundreds of sources, each subject to change, the EDW must not be keyed by whatever a subset of systems happens to use today.

Input factors for defining the business keys — business first, technical second

FIRST — business-side inputs	SECOND — technical-side inputs
Business process designs · business interviews · existing data marts · business glossary · semantic / logical / information models · MDM artifacts · industry reference model · fact/dim matrix	Technical designs · source databases · source metadata · application system guides & manuals · actual source system data

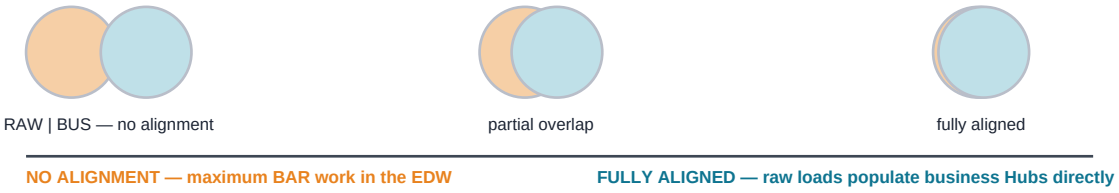
Raw keys vs business keys — aligned through a Link

The vault must be a true auditable mirror of the sources (raw keys, loaded as-is) AND be integrated around the EWBKs the business actually uses. When a raw key (“Person”) does not mean the same thing as the business key (“Customer”), the two get separate Hubs and the alignment is recorded in a Link — a Same-As-style structure (compare Ch 2 §1.2.2):



Example business rule: a Person becomes a Customer only when involved in a completed, non-cancelled Sale with a non-zero purchase price. The raw, auditable load goes to HUB_PERSON; the business-aligned load to HUB_CUSTOMER is generated by that rule — so it is flagged record_source = SYSGEN.

How much alignment work the EDW must do



■ BAR — Business key Alignment Rules

The business logic that aligns raw keys with EWBKs is called the BAR component, a subset of the Business Data Vault (BDW / BDV). Hultgren calls these rules “the heart” of the DV EDW: the primary objective of the warehouse is to integrate and historise data from many sources, and key alignment is where that integration actually happens. Note these transformations cannot be deferred to mart staging (not persisted) or the marts themselves (departmental, not enterprise-wide) — deferring them recreates the data-silo problem the EDW exists to solve.

Ch 6

Link Rules — 5.0

5.1 — A Link Must Contain Two or More Hub Keys

A Link joins Hubs together. It must import at least two Hub primary keys. A Link can never depend on another Link — only on Hubs.

Rule	Correct Design	Incorrect Design
Minimum keys	LINK_ORDER imports HUB_CUSTOMER key, HUB_PRODUCT key, and HUB_ORDER key — three Hubs	A table with only one Hub key — that is a Satellite, not a Link
Dependency rule	LINK_ORDER depends only on HUB_CUSTOMER and HUB_PRODUCT (both are Hubs)	A Link that imports another Link's primary key — Links cannot chain to other Links

5.2 — Hierarchical Link Uses Same Hub Twice

For parent-child relationships within the same entity type, the same Hub key is imported twice — once as the parent and once as the child.

Column	Example Value	Role
parent_product_hash_key	HK-bundle-A	FK to HUB_PRODUCT — represents the parent product bundle
child_product_hash_key	HK-item-001	FK to HUB_PRODUCT again — represents the child item within the bundle
load_date_timestamp	2024-06-01	When this parent-child relationship was first observed

5.3 — Link Load Date is an Attribute, Never Part of Primary Key

The Link load date records when this relationship was first observed. It is never part of the Link primary key — the PK is the composite of all imported Hub keys.

Primary Key Design	Correct?
All imported Hub hash keys combined — e.g. hub_customer_hk + hub_product_hk + hub_order_hk	Yes — the unique combination of business keys identifies the relationship
The same composite plus load_date_timestamp added to the PK	No — this would allow the same relationship to appear multiple times with different load dates, destroying the uniqueness guarantee

5.4 — Link Composite Key Must Be Unique

The combination of all imported Hub keys forms the Link primary key. This composite must be unique — one row per unique combination of business keys, valid for ALL time.

Customer Key	Product Key	Order Key	Rows in Link	Valid?
HK-cust01	HK-prod05	HK-ord99	1 row	Yes — first and only time this combination was seen
HK-cust01	HK-prod05	HK-ord99	2nd row with identical keys	No — exact same relationship already exists. Do not insert again

5.5 — Link Surrogate Key is Optional

If the composite key is too large or the database performs poorly with multi-column natural keys, a surrogate hash key can replace them as the single primary key.

Design	Primary Key	When to Use This Design
Without surrogate	All imported Hub hash keys combined as composite PK	Preferred when the database handles composite keys efficiently
With surrogate hash key	One single link_hash_key column as the entire PK	Use when composite key is too large or query performance suffers
Non-historized Links	No surrogate needed at all	Transactional immutable links with no child Satellites never need a surrogate

5.6 and 5.10 — Link Granularity

A Link's granularity is determined by the number and nature of Hub keys it imports. Always model at the LOWEST level of granularity for maximum tracking capability.

Number of Imported Hub Keys	What the Link Represents	Granularity Level
2 keys: Customer plus Order	Every unique customer-order pair	Coarse — cannot distinguish which products were in the order
3 keys: Customer plus Product plus Order	Every unique customer-product-order combination	Finer — can track individual product lines within an order
4 keys: Customer plus Product plus Order plus Store	Every unique combination of all four entities	Finest — maximum tracking detail across all relevant dimensions

5.7 — A Link Can Represent a Transaction, Hierarchy, or Relationship

Links are flexible — the same structure serves three distinct business purposes.

Link Type	Hub Keys Imported	Example
Relationship	Customer plus Product keys	Which customers have ever bought which products
Transaction	Customer plus Product plus Order plus Date keys	A specific completed purchase event at a point in time
Hierarchy	Same Hub key imported twice as parent and child	Product Item belongs to Product Bundle — both from HUB_PRODUCT

5.9 and 5.11 — Links Are Never Temporal and Represent ONE Instance For All Time

Links never contain begin or end dates. A Link records that a relationship EXISTS — not WHEN it was active. Temporality belongs in Effectivity Satellites on the Link. One unique key combination equals one row forever.

What Goes In the Link	What Goes in the Effectivity Satellite
The fact that Customer 001 and Product ABC were ever related — that the connection EXISTS	WHEN that relationship was active — effective dates, end dates, deletion markers
First observed date as a non-PK attribute only	Whether the relationship was active in January, inactive in February, and reactivated in March

5.13 — Link Driving Key Rule

The driving key is the consistent owner of the relationship — the part that stays constant while other parts of the relationship change.

Scenario	Driving Key	Non-Driving Key (the part that changed)
Salesperson reassigned from Account A to Account B	Account — it stays constant, the account relationship drives the link	Salesperson — it changed, the salesperson FK is the non-driving key
Customer order updated from Product X to Product Y	Customer plus Order — the order itself is constant	Product — it changed. New product FK on the same order number

Ch 7

Satellite Rules — 6.0

6.1 — Satellite Must Import One Parent Hub or Link Key

A Satellite cannot generate its own primary key. It must borrow the PK from its one and only parent Hub or Link. It can never create its own surrogate or hash key.

Valid Satellite PK	Invalid Satellite PK
hub_customer_hash_key (from parent Hub) plus load_date_timestamp — composite PK formed from parent	A satellite_sequence_id auto-generated by the Satellite itself — Satellites can never own their own key

6.2 — One Parent Only — Never Snowflaked

A Satellite can only attach to ONE parent Hub or Link. It can never chain to another Satellite. Satellites are never snowflaked.

Valid Structure	Invalid Structure
SAT_CUSTOMER_CONTACT attaches directly to HUB_CUSTOMER only — single parent	SAT_CUSTOMER_CONTACT attaches to SAT_CUSTOMER_DEMOGRAPHICS which attaches to HUB_CUSTOMER — this snowflaking is strictly forbidden in Data Vault

6.3 — Load Date Time Stamp is Part of Satellite Primary Key

Unlike Hubs and Links where load date is just an attribute, in a Satellite the load date IS part of the primary key. This allows multiple historical versions to coexist in the same table.

Primary Key Components	Uniquely Identifies
hub_customer_hash_key + load_date_timestamp = (HK-a3f9, 2024-01-01 08:00:00)	Version of customer data that was loaded on 1 January 2024
hub_customer_hash_key + load_date_timestamp = (HK-a3f9, 2024-06-01 08:00:00)	Updated version loaded on 1 June 2024 — email address changed
Both rows coexist in the same Satellite table simultaneously	Full history preserved — original row never deleted or overwritten

6.4 — Sub-Sequence is Optional

For situations where multiple rows arrive at the exact same microsecond — real-time feeds or multi-active satellites — a sub-sequence identifier is added to the primary key to guarantee uniqueness.

When Sub-Sequence Is Needed	Sub-Sequence Example Values
Two updates to the same customer arrive at exactly 14:32:05.000 — identical load timestamps would create a PK collision	Sub-sequence 1 for the first update, sub-sequence 2 for the second — same timestamp but distinct sequence resolves the collision
Multi-Active Satellite with 3 simultaneous addresses at the same load time	Sub-sequence 1 for Home, 2 for Work, 3 for Postal — all same load timestamp but different sequence values guarantee uniqueness

6.5 — Record Source is Required in Every Satellite

Every Satellite row must record which source system provided the data. This ensures complete row-level traceability from the vault back to its origin.

Satellite Row	Record Source Value	What It Tells You
Customer contact row from January 2024	CRM_SALESFORCE	This version of the email and phone came from Salesforce CRM
Customer contact row updated June 2024	MANUAL_CORRECTION	An analyst manually corrected the email address on this date
Customer demographic row from onboarding	ONBOARDING_PORTAL	This data was entered by the customer themselves during signup

6.6 — Must Have at Least One Descriptive Element

A Satellite must contain at least one descriptive attribute about its parent. A Satellite containing ONLY the parent key, load date, and record source provides no useful context and is not a valid Satellite.

Valid Satellite	Invalid Satellite
SAT_CUSTOMER_CONTACT contains: hub_customer_hash_key, load_date, record_source, email_address, phone_number, postal_address	A Satellite with ONLY hub_customer_hash_key, load_date_timestamp, and record_source — no descriptive attributes at all. It describes nothing about the customer

6.7 — Business Vault Satellite Can Contain Computed Attributes

In the Business Vault layer, a Satellite can contain attributes derived by soft business rules — calculated fields, aggregates, or standardised values. These are NOT raw source data.

Layer	Satellite Type	Example Contents
Raw Vault	Standard Satellite (SAT)	Raw email and phone exactly as received from the source system — no transformation
Business Vault	Business Satellite (BSAT)	Standardised phone number in E.164 format, validated email, customer lifetime value calculated by a business rule

6.8 — Satellites Store Data Over Time — No Physical Updates Ever

A Satellite's core purpose is to be a time-series store. New rows are always INSERTED — never updated or deleted. This guarantees a complete and immutable audit trail.

Business Event	What Happens in the Satellite
Customer changes email address on 1 June	New row inserted: (HK-cust01, 2024-06-01, new_email). The old row with the original email still exists untouched
Customer changes email again on 15 June	Another new row inserted. All three versions coexist in the same table
Customer is deleted in the source system	New row inserted in Effectivity Satellite with a deleted flag. Original data rows in the descriptive Satellites are never touched

6.9 — Reference Table Access via Natural Key Only

A Satellite may reference a code table using a natural key (the actual code value). Physical foreign keys to reference tables are NEVER added to the physical model — references are implied and resolved at query time only.

Column Stored in Satellite	How Reference Is Resolved at Query Time
payment_method_code = 'CC' stored in Satellite	At query time join to REF_PAYMENT_METHOD where method_code = 'CC' to get the description 'Credit Card'. No FK physically defined in the model
state_code = 'NSW' stored in Satellite	At query time join to REF_STATE where state_code = 'NSW' to get 'New South Wales'. The join is implied, not physically enforced

6.10 — Satellites Can Be Split or Merged at Any Time

A Satellite can be restructured at any time — as long as NO historical data and NO audit trail is lost during the process.

Action	When to Do It	Non-Negotiable Condition
Split a Satellite into two	One Satellite has both fast-changing (email, phone) and slow-changing (date of birth) attributes causing unnecessary row proliferation	ALL historical rows must be migrated to the two new Satellites before the original Satellite is dropped — no history can be lost
Merge two Satellites into one	Two Satellites covering the same attributes from different source systems are being consolidated into a single unified structure	ALL historical rows from both original Satellites must exist in the merged Satellite — complete history from both sources must be preserved

Ch 8

Naming Conventions — 7.0

Naming conventions in Data Vault are essential for automation and standardisation. Without consistent naming ETL tools cannot auto-generate code and models become unmanageable as they grow. You must document your chosen convention. The prefixes and suffixes below are recommendations, not requirements.

7.1 Entity Naming Conventions

Entity Type	Prefix or Suffix Options	Example Table Names
Hub	H, HUB, HB	HUB_CUSTOMER, H_PRODUCT, HB_ORDER
Link	L, LINK, LNK	LINK_ORDER_ITEM, LNK_SALE, L_CUSTOMER_PRODUCT
Satellite	S, SAT	SAT_CUSTOMER_CONTACT, S_PRODUCT_DETAILS
Stage	STG	STG_CRM_CUSTOMER, STG_ORDER_LINES
Hierarchical Link	HL, HLNK, HLINK	HLNK_PRODUCT_BUNDLE, HL_ORG_CHART
Same-As Link	SAL, SALNK, SLNK	SAL_CUSTOMER_MASTER, SALNK_PRODUCT_SYNONYM
Point-in-Time	PIT, PT	PIT_CUSTOMER, PT_PRODUCT_HISTORY
Bridge	B, BRDG, BRG	BRDG_ORDER_CUSTOMER, B_SALE_BASE
Business Hub	BH, BHUB	BHUB_CUSTOMER_CLEANSED, BH_PRODUCT_STANDARD
Business Link	BL, BLNK, BLINK	BLINK_ORDER_ENRICHED, BL_SALE_RULES
Business Satellite	BS, BSAT, BST	BSAT_CUSTOMER_SCORED, BS_PRODUCT_VALUED
View	V	V_CUSTOMER_CURRENT, V_PRODUCT_ACTIVE
View Dimension	VDIM, VD	VDIM_CUSTOMER, VD_PRODUCT
View Fact	VF, VFCT	VF_SALES, VFCT_REVENUE
Fact	FCT, FACT, F	FCT_SALES, FACT_REVENUE, F_ORDER
Dimension	D, DIM	DIM_CUSTOMER, D_PRODUCT, DIM_DATE
Report Collection	RPT, RC	RPT_MONTHLY_SALES, RC_EXECUTIVE_DASHBOARD

7.2 Field Naming Conventions

Attribute Type	Prefix or Suffix Options	Example Field Names
Record Source	R, RSRC, RS	customer_RSRC, order_RS, product_RSRC
Sequence ID	SQN, SEQ	row_SQN, line_SEQ
Date Time Stamps	DTS, DTM	transaction_DTS, event_DTM
Date Stamps	DS, DT	effective_DS, order_DT
Time Stamps	TMS, TS, TM	created_TMS, updated_TS

Attribute Type	Prefix or Suffix Options	Example Field Names
Load Date Time Stamp	LDTS, LDDTS, LDTM	customer_LDTS, product_LDTM
User Watch Fields	USR, U	dba_USR, modified_U
Occurrence Numbers	OCC, OCNUM, ONUM	address_OCC, phone_OCNUM
Load End Date Time Stamps	LEDTS	record_LEDTS (deprecated — legacy reference only)
Sub Sequence	SSQN, SSQ, SUBSQN	address_SSQN, event_SSQ
Applied Dates	APPDT, ADT, APDT	transaction_APPDT, backdate_ADT
Hash Keys	HK, HashKey, HKEY	customer_HK, product_HKEY, order_HashKey
Hash Differences	HD, HashDiff, HDIFF, HDF	contact_HD, demographics_HDIFF, details_HDF

■ Remember — Documentation Is Mandatory

These are RECOMMENDATIONS not requirements. What IS required is that you create and maintain a naming convention document for your specific Data Vault model. Consistent naming is what enables ETL automation — without it every Hub, Link, and Satellite must be hand-coded rather than auto-generated.

Ch 9

Deprecated Rules — 10.0

These rules were part of earlier versions of Data Vault but have been **removed from Data Vault 2.0**. They should never be used in new designs. Big data volumes, platform diversity, and real-time requirements made them unworkable.

10.1 Sequence ID Surrogate Keys — DEPRECATED

Auto-incrementing integer surrogate keys (IDENTITY columns) were the original DV 1.0 primary key approach. They have been replaced by Hash Keys in Data Vault 2.0.

Problem	Why It Fails	Hash Key Solution
Problem with Sequence IDs	Why It Fails at Scale	Hash Key Solution
Row-by-row lookup required	To insert a child row you must first look up the parent sequence ID. This is sequential and non-parallelisable — cannot scale to terabyte volumes	Hash is computed directly from the business key. No lookup needed — any ETL process computes it independently
Non-deterministic	Two ETL processes running in parallel may assign different sequence IDs to the same business key — creating duplicates or conflicts	MD5 of CUST-10045 always produces the same hash everywhere — deterministic by definition, no conflicts possible
Cross-geography problems	Sequence IDs from a US database conflict with sequence IDs from an Australian database when integrating across regions	Hash of the same business key is identical everywhere in the world — no geographic or regional conflict ever
Privacy compliance	Sending clear-text business keys across country borders for lookup violates data sovereignty and privacy regulations in many jurisdictions	Hash the key locally — send only the hash across the boundary, not the clear-text business key itself

DEPRECATED — Do Not Use in Data Vault 2.0

NEVER use auto-increment sequence IDs as primary keys in Data Vault 2.0. Use Hash Keys (MD5, SHA-256) computed from business keys instead.

10.2 Satellite Load End-Dates — DEPRECATED

In DV 1.0 each Satellite row had a load_end_date column that was physically UPDATED when a newer version arrived. This required UPDATE statements which do not scale to petabyte data volumes.

Old Approach	Problem	DV 2.0 Solution
Old DV 1.0 Approach	Problem with This Approach	DV 2.0 Solution
Satellite row has a load_end_date column	When a new version arrives the old row's load_end_date must be physically UPDATED. Not all platforms support this at scale. Breaks insert-only architecture	No load_end_date stored in the Raw Vault at all. Only INSERT operations ever occur — no updates
End dates maintained directly in the raw Satellite	Out-of-order data arrival causes incorrect end dates. Real-time feeds frequently arrive out of sequence creating wrong end date values	Use SQL LEAD or LAG window functions to compute end dates at query time when needed
Required to identify the current active record	Queries must filter where load_end_date equals the high date to find the current row	Compute end dates in PIT and Bridge tables in the Information Mart. Raw Vault Satellites remain insert-only

DEPRECATED — Do Not Use in Data Vault 2.0

NEVER add a load_end_date column to Raw Vault Satellites. Compute end dates using LEAD or LAG window functions and materialise them in PIT or Bridge tables in the Information Mart if needed.

10.3 Hub and Link Last Seen Dates — DEPRECATED

In early versions Hub and Link tables had a last_seen_date column that was physically UPDATED each time the business key or relationship was observed again. Like end-dates, this requires UPDATE statements that do not scale.

Old Approach	Problem	DV 2.0 Solution
Old Approach	Problem	DV 2.0 Solution
Hub has a last_seen_date column updated on every load run	Physical UPDATE on Hub rows. Breaks parallelism. Not supported by all platforms. Does not scale to hundreds of terabytes	Use Effectivity Satellites to track when a business key was active or last observed
Link has a last_seen_date column updated on every load run	Same scalability problem. Out-of-order data can set last_seen_date to an incorrect earlier date, corrupting the timeline	Use Effectivity Satellites on the Link or query the Satellite history to determine the last observation date

DEPRECATED — Do Not Use in Data Vault 2.0

NEVER add last_seen_date columns to Hubs or Links in Data Vault 2.0. Use Effectivity Satellites to track temporal activity instead.

Core DV 2.0 Principle: INSERT-ONLY Architecture

All three deprecated patterns share the same root cause: they require physical UPDATE statements on existing rows. Data Vault 2.0 is an INSERT-ONLY architecture. No row in the Raw Vault is ever physically modified after it is inserted. This is what makes it fully auditable, parallelisable, and scalable to any data volume.